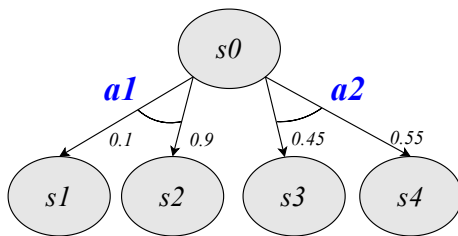


AND/OR Heuristic Search

- AND/OR Heuristic Search⁵ can solve **non-deterministic** state-space search problems;
 - Each possible outcome occurs with some **probability** $P(s'|s, a)$;
- AND/OR Heuristic Search⁶ is usually used to solve:
 - *Stochastic Shortest-Path problems*, MDP problems with terminal states;
 - FOND (Fully-Observable Non-Deterministic) planning problems;

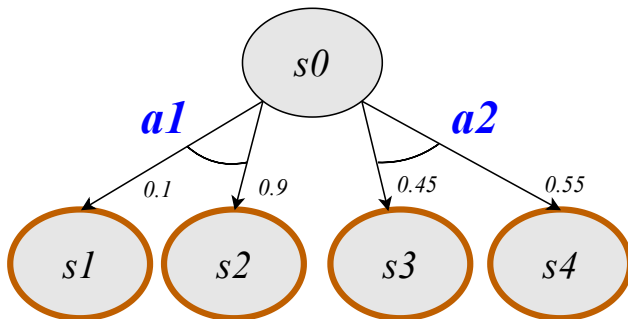


⁵ Martelli, A. and Montanari, U. 1978. *Optimizing Decision Trees Through Heuristically Guided Search*. *Communications of the ACM* 21(12):1025-1039.

⁶ Nilsson, N.J. 1980. *Principles of Artificial Intelligence*.

AND/OR Graphs

- An **OR Node** is the choice of an **action** at the state;
- An **AND Node** is the **outcomes** of an action;



- AO* is a **best-first search** algorithm similar to A*, but for (**acyclic**) AND/OR Graphs;
- AO* solves a state-space search problem by gradually building a solution graph G' ;
- **Input:** A non-deterministic state-space search problem;
- **Output:** A solution that has a conditional structure (a **policy**), or more generally, an **acyclic graph**.

- **Expanding Stage:** Expand one or more nodes on the fringe of the best solution graph;
- **Cost Revision Stage:** Revise and propagate the new values on the fringe using a *backward induction*;

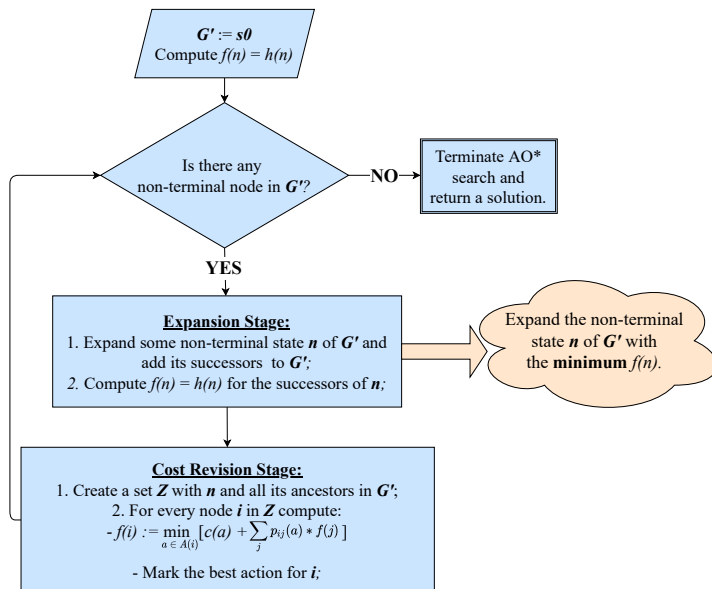
AO* Search: Expanding Stage

- Expand one or more non-terminal nodes of the best solution graph G' , namely;
 - The non-terminal node in G' with the **minimum** $f(s) = h(s)$;

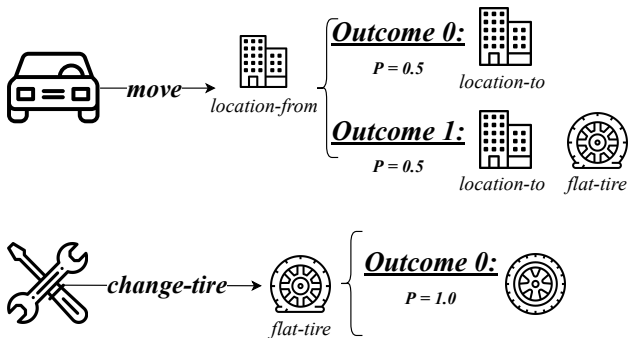
AO* Search: Cost Revision Stage

- Revise and propagate the new values on the fringe using a *backward induction*;
 - ① **Update** the *evaluation function* $f(n)$ for the expanded node n and its ancestors in G' .
 - $Z = n$ and its ancestors in G' :
$$\forall i \in Z \ f(i) := \min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$
 - ② **Mark the best action** for every OR node in G' according to $f(n)$;

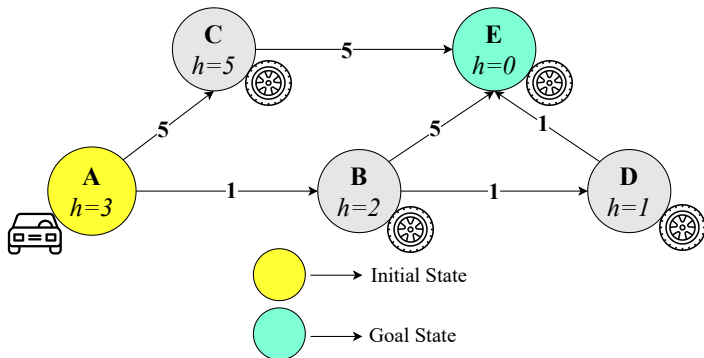
AO* Search Algorithm



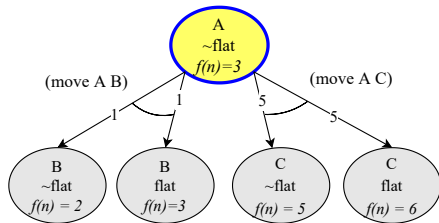
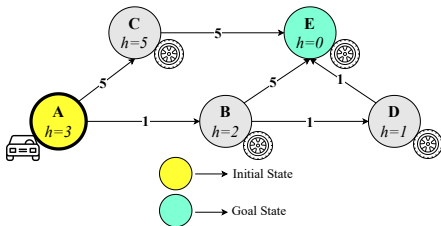
AO* Search Example (1 of 5)



AO* Search Example (2 of 5)



AO* Search Example (3 of 5)



- $G' = \langle [A, \neg f] \rangle$
- **Expansion:** $n = \langle [A, \neg f] \rangle$
- **Cost Revision:** $Z = n$ and its ancestors in G'

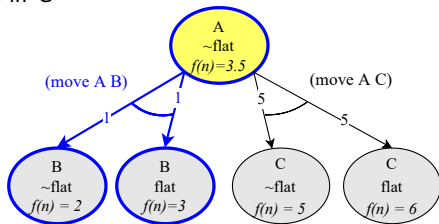
$$\min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$

- (move A B):

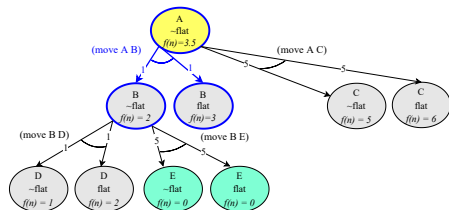
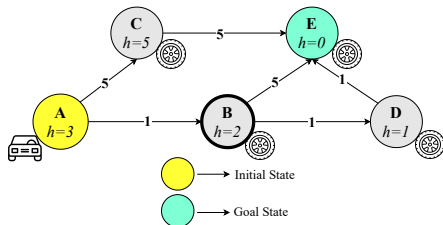
$$1 + [(0.5*3) + (0.5*2)] = 3.5$$

- (move A C):

$$5 + [(0.5*5) + (0.5*6)] = 11.5$$



AO* Search Example (4 of 5)



- $G' = \langle [A, \neg f], [B, \neg f], [B, f] \rangle$

- Expansion:** $n = \langle [B, \neg f] \rangle$

- Cost Revision:** $Z = n$ and its ancestors in G'

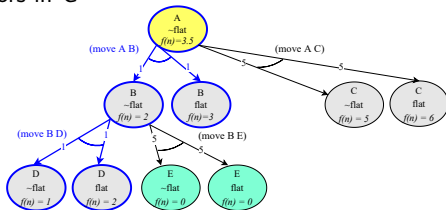
$$\min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$

- (move B D):

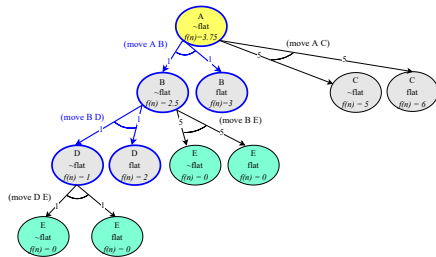
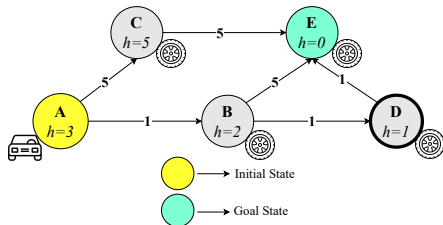
$$1 + [(0.5*1) + (0.5*2)] = 2.5$$

- (move B E):

$$5 + [(0.5*0) + (0.5*0)] = 5$$



AO* Search Example (5 of 5)



• $G' = \langle [A, \neg f], [B, \neg f], [B, f], [D, \neg f], [D, f] \rangle$

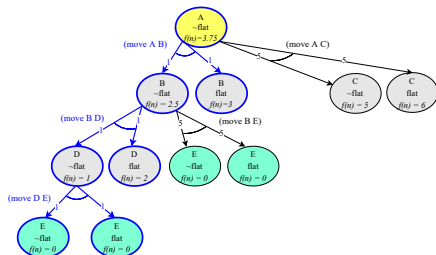
• **Expansion:** $n = \langle [D, \neg f] \rangle$

• **Cost Revision:** Z

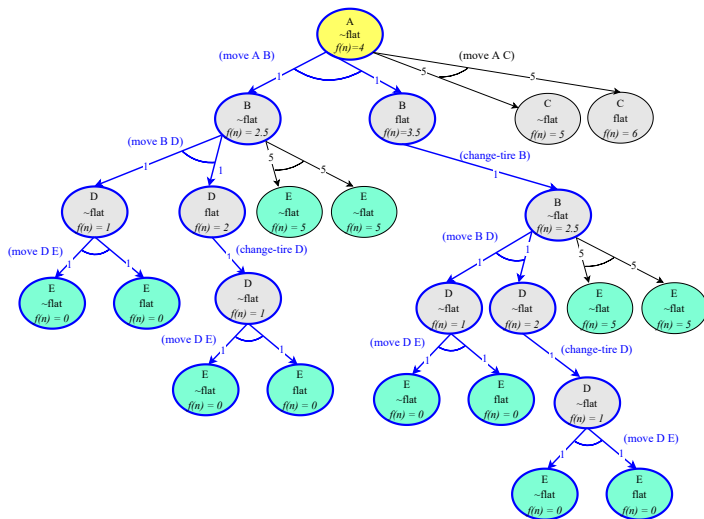
$$\min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$

- (move D E):

$$1 + [(0.5 * 0) + (0.5 * 0)] = 1$$



AO* Search Example - Solution



Theoretical Properties of AO* Search

Theorem (Termination and Completeness)

AO* search algorithm is complete and terminate if:

- 1 The branching factor is finite; and
- 2 The action costs are non-negative.

Theorem (Optimal Efficiency)

AO* search algorithm is optimal if:

- 1 **Admissibility:** $h(s)$ must be an admissible heuristic; and
- 2 **Consistency:** h must be consistent (monotone).

- The complexity of the **Cost Revision Stage** is linear in the number of states.

AND/OR Search vs. Policy Iteration and Value Iteration

- AND/OR Search algorithms **do not explore the entire state-space**;
 - However, we need a **“good”** heuristic function.
- Policy Iteration and Value Iteration⁷ **explore the entire state-space**;

⁷ Algorithms use for solving MDPs.